Data Mining

Language Mining

https://data-mining.github.io/winter-2026/

CS 453/553 – Winter 2026
Yu Wang, Ph.D.
Assistant Professor
Computer Science
University of Oregon
Slide Courtesy: Zhiting Hu from UCSD

Where we are?

	03/02/2026 00:00 Monday	Presentation 6	**Group** ○ Group 11: 7-7:15 pm ○ Group 12: 7:15-7:30 pm ○ Zoom
Lecture	03/02/2026 Monday	Language Mining	**Course Materials:** ○ Slides
	03/04/2026 00:00 Wednesday	Presentation 6	**Group** ○ Group 13: 7-7:15 pm ○ Group 14: 7:15-7:30 pm ○ Zoom
Lecture	03/04/2026 Wednesday	Language Mining 2	**Course Materials:** ○ Slides
	03/09/2026 00:00 Monday	Presentation 6	**Group** ○ Group 15: 7-7:15 pm ○ Group 16: 7:15-7:30 pm ○ Zoom
Lecture	03/09/2026 Monday	Review Future	**Course Materials:** ○ Slides
Exam	**03/11/2026 01:20 Wednesday**	**Quzz 2**	Topics: ○ Lecture 9 - Lecture 16 ○ Closed Book
Due	**03/20/2026 23:59 Friday**	**Project Report Due**	

Language Mining

Final Week

Fun Facts

Our agreement with the Department of War

Listen to article | 7:59 Share

Yesterday we reached an agreement with the Pentagon for deploying advanced AI systems in classified environments, which we requested they also make available to all AI companies.

We think our agreement has more guardrails than any previous agreement for classified AI deployments, including Anthropic's. Here's why.

We have three main red lines that guide our work with the DoW, which are generally shared by several other frontier labs:

- No use of OpenAI technology for mass domestic surveillance.
- No use of OpenAI technology to direct autonomous weapons systems.
- No use of OpenAI technology for high-stakes automated decisions (e.g. systems such as "social credit").

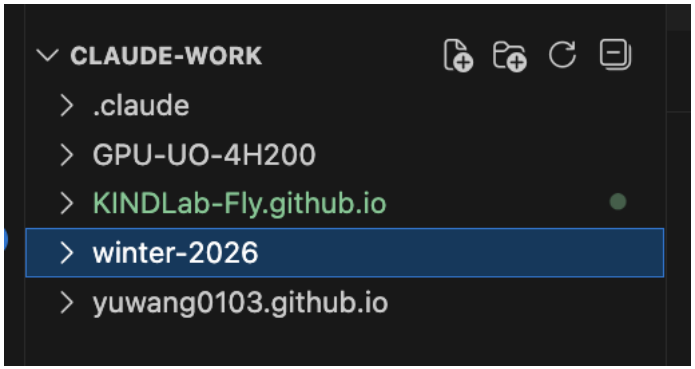
Announcements Policy

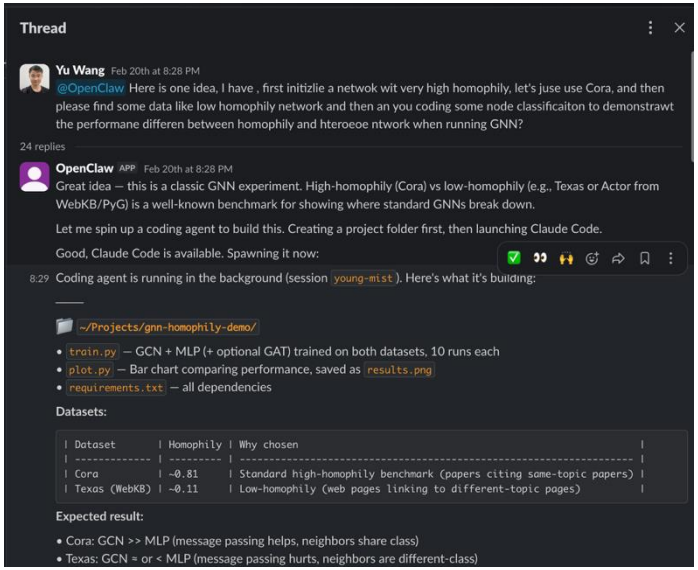
Statement from Dario Amodei on our discussions with the Department of War

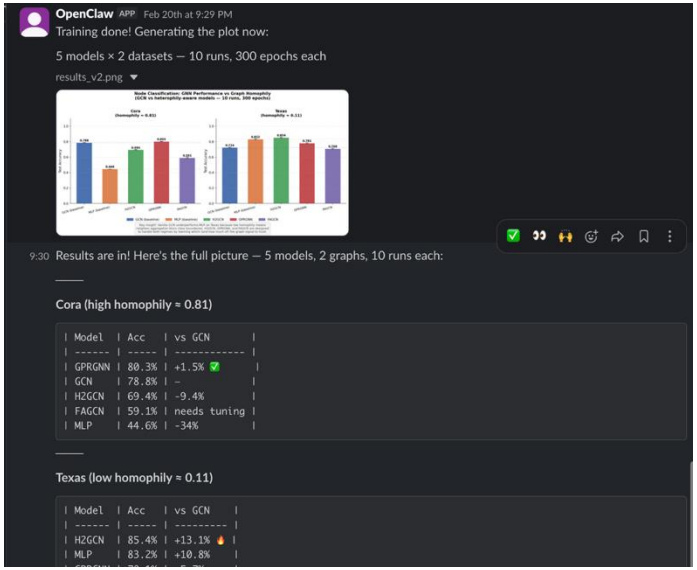
Feb 26, 2026

I believe deeply in the existential importance of using AI to defend the United States and other democracies, and to defeat our autocratic adversaries.

Anthropic has therefore worked proactively to deploy our models to the Department of War and the intelligence community. We were the first frontier AI company to deploy our models in the US government's classified networks, the first to deploy them at the National Laboratories, and the first to provide custom





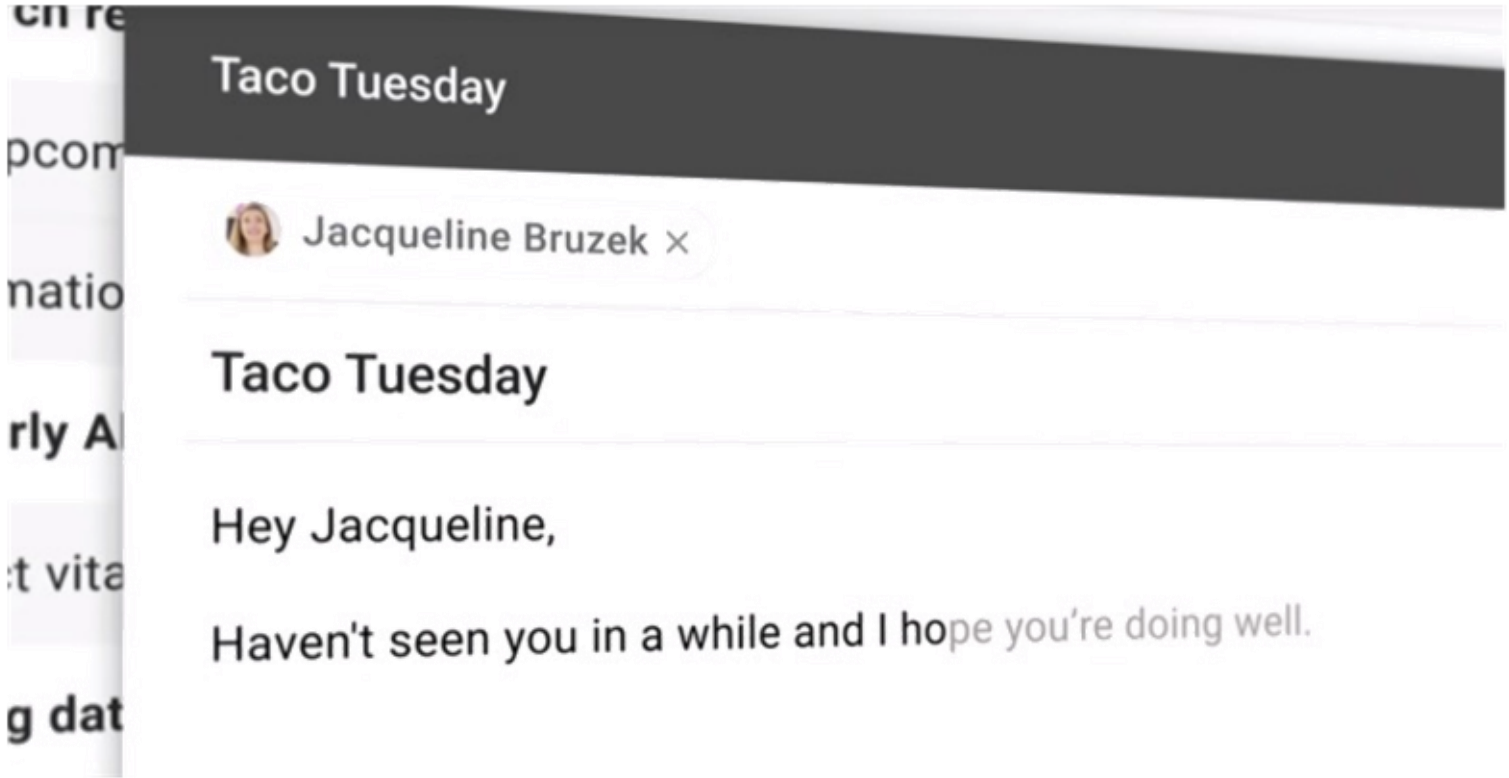


Summary so far

- language models utilities
  - Generation, evaluation of fluency, few-shot prediction (GPT3), …
- N-gram language models
  - Unigram LM
  - N-gram LM
- Neural language models:
  - Embedding: one-hot vectors -> embedding vectors
  - Neural networks

Motivations of Language Models

- Generation



Email auto-completion

Motivations of Language Models

- Generation

Title: United Methodists Agree to Historic Split
Subtitle: Those who oppose gay marriage will form their own denomination
Article: After two days of intense debate, the United Methodist Church has agreed to a historic split - one that is expected to end in the creation of a new denomination, one that will be "theologically and socially conservative," according to The Washington Post. The majority of delegates attending the church's annual General Conference in May voted to strengthen a ban on the ordination of LGBTQ clergy and to write new rules that will "discipline" clergy who officiate at same-sex weddings. But those who opposed these measures have a new plan: They say they will form a separate denomination by 2020, calling their church the Christian Methodist denomination.
The Post notes that the denomination, which claims 12.5 million members, was in the early 20th century the "largest Protestant denomination in the U.S.," but that it has been shrinking in recent decades. The new split will be the second in the church's history. The first occurred in 1968, when roughly 10 percent of the denomination left to form the Evangelical United Brethren Church. The Post notes that the proposed split "comes at a critical time for the church, which has been losing members for years," which has been "pushed toward the brink of a schism over the role of LGBTQ people in the church." Gay marriage is not the only issue that has divided the church. In 2016, the denomination was split over ordination of transgender clergy, with the North Pacific regional conference voting to ban them from serving as clergy, and the South Pacific regional conference voting to allow them.

Figure 3.14: The GPT-3 generated news article that humans had the greatest difficulty distinguishing from a human written article (accuracy: 12%).

Motivations of Language Models

- Evaluation of language fluency
  - Ex:

    Successful speech recognition requires generating a word sequence that is:

    - Faithful to the acoustic input
    - Fluent

    If we're mapping acoustics $\boldsymbol{a}$ to word sequences $\boldsymbol{w}$, then:

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{\operatorname{argmax}} \operatorname{Faithfulness}(\boldsymbol{w}; \boldsymbol{a}) + \operatorname{Fluency}(\boldsymbol{w})$$

    Language models can provide a "fluency" score.

Motivations of Language Models

- Evaluation of language fluency
  - Ex: Other tasks that have text (or speech) as output:
    - translation from one language to another
    - conversational systems
    - document summarization
    - image captioning
    - optical character recognition
    - spelling and grammar correction

  If we're mapping inputs $\boldsymbol{i}$ to word sequences $\boldsymbol{w}$, then:

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{\operatorname{argmax}} \operatorname{Faithfulness}(\boldsymbol{w}; \boldsymbol{i}) + \operatorname{Fluency}(\boldsymbol{w})$$

  Language models can provide a "fluency" score.

Motivations of Language Models

- Few-shot prediction (e.g., GPT3)

Prompts in Red. GPT3 responses in Blue

Q. How do you draw a bicycle?
A. You draw a bicycle with two circles connected by a line.

Q. How do you draw a ball?
A. You draw a ball with one circle.

Q. How do you draw a house?
A. You draw a house with a trapezium on top of a square.

Q. How do you draw a rabbit?
A. You draw a rabbit with four circles: one large circle for the head, a small one for the nose, a large circle for the body, and two tiny circles for the feet.

Q. How do you draw a snail?
A. You draw a snail with six circles: one large circle for the shell, two small circles for the antennae, one large circle for the head, one small circle for the tail, and a small circle for the mouth.

Notations

- $\mathcal{V}$ is a finite set of (discrete) symbols (words or characters); $V = |\mathcal{V}|$
- $\mathcal{V}^*$ is the (infinite) set of sequences of symbols from $\mathcal{V}$
- In language modeling, we imagine a sequence of random variables $X_1, X_2, \ldots$ that continues until some X_n takes the value "⬡" (a special end-of-sequence symbol).
- $\mathcal{V}^\dagger$ is the (infinite) set of sequences of $\mathcal{V}$ symbols, with a single ⬡, which is at the end.

The Language Modeling Problem

- Input: training data $\boldsymbol{x} = (x_1, x_2, \ldots, x_N)$ in $\mathcal{V}^\dagger$
  - (assuming one instance $\boldsymbol{x}$ for simplicity of notations)
- Output: p: $\mathcal{V}^\dagger \rightarrow \mathbb{R}$
- Think of p as a measure of plausibility

Probabilistic Language Model

- We let p be a probability distribution, which means that

$$\forall \boldsymbol{x} \in \mathcal{V}^{\dagger}, p(\boldsymbol{x}) \geq 0$$

$$\sum_{\boldsymbol{x} \in \mathcal{V}^{\dagger}} p(\boldsymbol{x}) = 1$$

- Advantages:
  - Interpretability
  - We can apply the maximum likelihood principle to build a language model from data

Decomposing using the Chain Rule

$$p(\boldsymbol{X} = \boldsymbol{x}) = \begin{pmatrix} p(X_1 = x_1) \\ \cdot\, p(X_2 = x_2 \mid X_1 = x_1) \\ \cdot\, p(X_3 = x_3 \mid \boldsymbol{X}_{1:2} = \boldsymbol{x}_{1:2}) \\ \vdots \\ \cdot\, p(X_N = \bigcirc \mid \boldsymbol{X}_{1:N-1} = \boldsymbol{x}_{1:N-1}) \end{pmatrix}$$

$$= \prod_{i=1}^{N} p(X_i = x_i \mid \boldsymbol{X}_{1:i-1} = \boldsymbol{x}_{1:i-1})$$

Example:

$\boldsymbol{x}$ = (*I, like, this, movie, …*)

$$p(\boldsymbol{x}) = \cdots p_\theta(like \mid I)\, p_\theta(this \mid I, like) \cdots$$

Predict each word based on the "history"

Summary so far

- language models utilities
  - Generation, evaluation of fluency, few-shot prediction (GPT3), …
- N-gram language models
  - Unigram LM
  - N-gram LM
- Neural language models:
  - Embedding: one-hot vectors -> embedding vectors
  - Neural networks

Unigram Model: Empty History

$$p(\boldsymbol{X} = \boldsymbol{x}) = \prod_{i=1}^{N} p(X_i = x_i \mid \boldsymbol{X}_{1:i-1} = \boldsymbol{x}_{1:i-1})$$

$$\overset{\text{assumption}}{=} \prod_{i=1}^{N} p(X_i = x_i; \boldsymbol{\theta}) = \prod_{i=1}^{N} \theta_{x_i}$$

Multinomial distribution

Maximum likelihood estimate: for every $v \in \mathcal{V}$,

$$\theta_v^* = \frac{\sum_{i=1}^{N} \mathbf{1}\{x_i = v\}}{N}$$

$$= \frac{\text{count}_{\boldsymbol{x}}(v)}{N}$$

Example

The probability of

Presidents tell lies .

is:

$$p(X_1 = \text{Presidents}) \cdot p(X_2 = \text{tell}) \cdot p(X_3 = \text{lies}) \cdot p(X_4 = .) \cdot p(X_5 = \bigcirc)$$

In unigram model notation:

$$\theta_{\text{Presidents}} \cdot \theta_{\text{tell}} \cdot \theta_{\text{lies}} \cdot \theta_{.} \cdot \theta_{\bigcirc}$$

Using the maximum likelihood estimate for $\boldsymbol{\theta}$, we could calculate:

$$\frac{\text{count}_{\boldsymbol{x}}(\text{Presidents})}{N} \cdot \frac{\text{count}_{\boldsymbol{x}}(\text{tell})}{N} \cdots \frac{\text{count}_{\boldsymbol{x}}(\bigcirc)}{N}$$

Unigram Models: Assessment

Pros:

- Easy to understand
- Cheap
- Good enough for information retrieval (maybe)

Cons:

- Fixed, known vocabulary assumption
- "Bag of words" assumption is linguistically inaccurate
  - $p(\text{the the the the}) \gg p(\text{I want ice cream})$

n-gram Models

$$p(\boldsymbol{X} = \boldsymbol{x}) = \prod_{i=1}^{N} p(X_i = x_i \mid \boldsymbol{X}_{1:i-1} = \boldsymbol{x}_{1:i-1})$$

$$\overset{\text{assumption}}{=} \prod_{i=1}^{N} p(X_i = x_i \mid X_{i-\mathsf{n}+1:i-1} = \boldsymbol{x}_{i-\mathsf{n}+1:i-1}; \boldsymbol{\theta})$$

$$= \prod_{i=1}^{N} \theta_{x_i | \boldsymbol{x}_{i-\mathsf{n}+1:i-1}}$$

$(\mathsf{n}-1)$th-order Markov assumption $\equiv$ n-gram model

- Unigram model is the $\mathsf{n} = 1$ case
- For a long time, trigram models ($\mathsf{n} = 3$) were widely used
- 5-gram models ($\mathsf{n} = 5$) were common in MT for a time

n-gram Models

- Maximum likelihood estimate for the n-gram model's probability of v given a $(n-1)$-length history $\boldsymbol{h}$

$$
\begin{aligned}
\theta_{v|\boldsymbol{h}} &= p(X_i = v \mid \boldsymbol{X}_{i-\mathsf{n}+1:i-1} = \boldsymbol{h}) \\
&= \frac{p(X_i = v, \boldsymbol{X}_{i-\mathsf{n}+1:i-1} = \boldsymbol{h})}{p(\boldsymbol{X}_{i-\mathsf{n}+1:i-1} = \boldsymbol{h})} \\
&= \frac{\mathrm{count}_{\boldsymbol{x}}(\boldsymbol{h}v)}{N} \Big/ \frac{\mathrm{count}_{\boldsymbol{x}}(\boldsymbol{h})}{N} \\
&= \frac{\mathrm{count}_{\boldsymbol{x}}(\boldsymbol{h}v)}{\mathrm{count}_{\boldsymbol{x}}(\boldsymbol{h})}
\end{aligned}
$$

Choosing n is a Balancing Act

If n is too small, your model can't learn very much about language.

As n gets larger:

- The number of parameters grows with $O(V^{\mathrm{n}})$.
- Most n-grams will never be observed, so you'll have lots of zero probability n-grams. This is an example of **data sparsity**.
- Your model depends increasingly on the training data; you need (lots) more data to learn to generalize well.

This is a beautiful illustration of the bias-variance tradeoff.

Other "tricks"

- Smoothing

  The game: prevent $\theta_{v|\boldsymbol{h}} = 0$ for any v and $\boldsymbol{h}$, while keeping $\sum_{\boldsymbol{x}} p(\boldsymbol{x}) = 1$ so that perplexity stays meaningful.

  - Simple method: add $\lambda > 0$ to every count (including counts of zero) before normalizing (the textbook calls this "Lidstone" smoothing)

- Dealing with Out-of-Vocabulary Terms
  - Define a special OOV or "unknown" symbol unk. Transform some (or all) rare words in the training data to unk.
  - Build a language model at the character level.
  - Some new methods use data-driven, deterministic tokenization schemes that segment some words into smaller parts to reduce the effective vocabulary size (Sennrich et al., 2016; Wu et al., 2016).

n-gram Models: Assessment

Pros:

- Easy to understand
- Cheap (with modern hardware; Lin and Dyer, 2010)
- Fine in some applications and when training data is scarce

Cons:

- Fixed, known vocabulary assumption
- Markov assumption is linguistically inaccurate
  - (But not as bad as unigram models!)
- Data sparseness problem

Summary so far

- language models utilities
  - Generation, evaluation of fluency, few-shot prediction (GPT3), …
- N-gram language models
  - Unigram LM
  - N-gram LM
- Neural language models:
  - Embedding: one-hot vectors -> embedding vectors
  - Neural networks

Neural Language Models

Instead of a lookup for a word and fixed-length history ($\theta_{v|\boldsymbol{h}}$), define a vector function:

$$p(X_i \mid \boldsymbol{X}_{1:i-1} = \boldsymbol{x}_{1:i-1}) = \mathbf{NN}(\mathbf{enc}(\boldsymbol{x}_{1:i-1}); \boldsymbol{\theta})$$

where $\boldsymbol{\theta}$ do the work of *encoding* the history and *transforming* it into a distribution over the next word.
The transformation is described as a composed series of simple transformations or "layers."

Neural Language Models

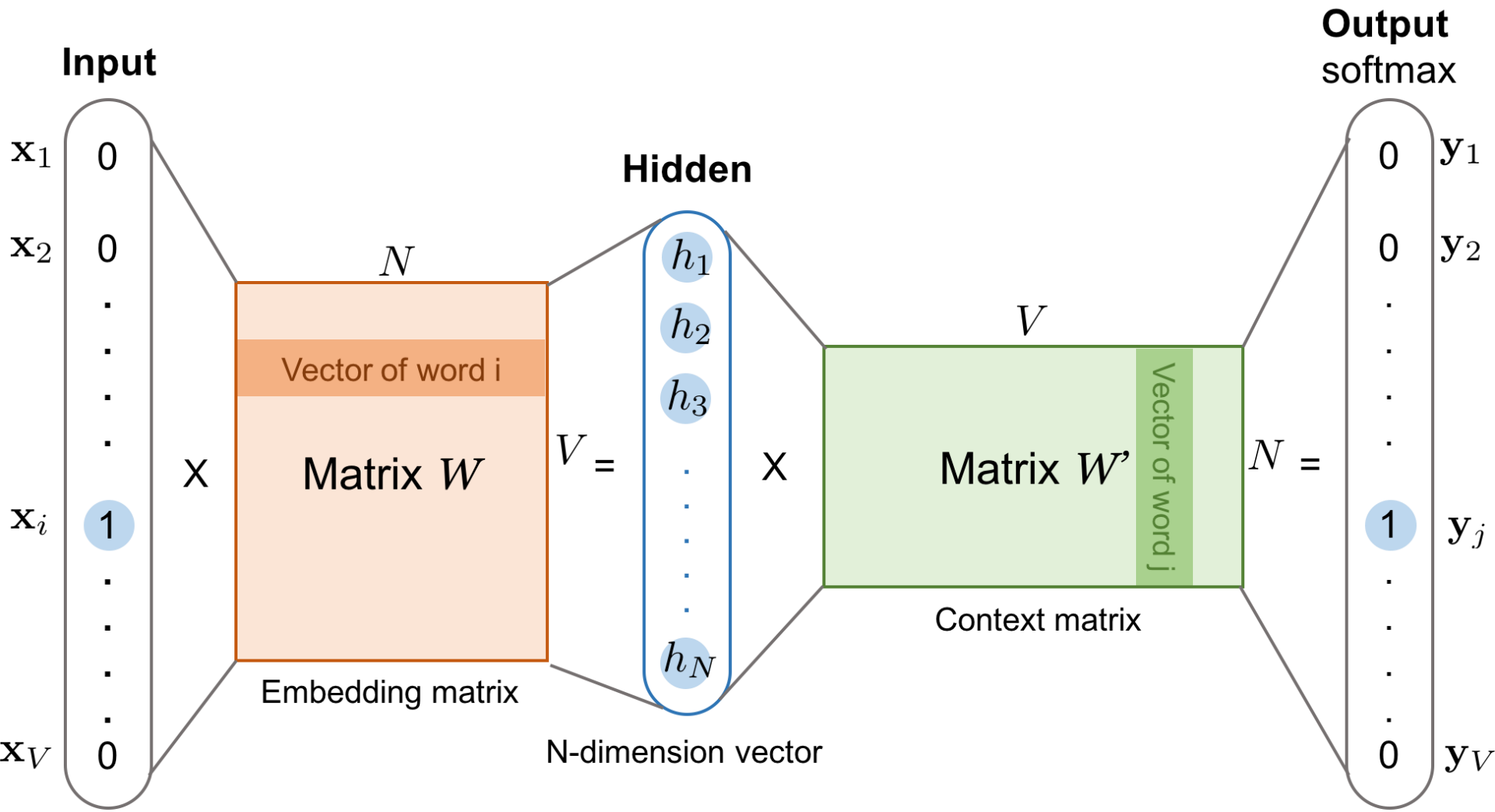
Instead of a lookup for a word and fixed-length history ($\theta_{v|\boldsymbol{h}}$), define a vector function:

$$p(X_i \mid \boldsymbol{X}_{1:i-1} = \boldsymbol{x}_{1:i-1}) = \mathbf{NN}(\mathbf{enc}(\boldsymbol{x}_{1:i-1}); \boldsymbol{\theta})$$

where $\boldsymbol{\theta}$ do the work of *encoding* the history and *transforming* it into a distribution over the next word.
The transformation is described as a composed series of simple transformations or "layers."

- We first map word histories $\boldsymbol{h}$ to vectors/matrices
- We interpret the output as $p(X_i \mid \boldsymbol{X}_{1:i-1} = \boldsymbol{h})$

Language Mining – Neural Language Models



Two Key Components

- "Embedding" words as vectors
- Layering to increase capacity (i.e., the set of distributions that can be represented).

"One Hot" Vectors

Let $\mathbf{e}_i \in \mathbb{R}^V$ be the ith column of the identity matrix $\mathbf{I}$.

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}; \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}; \quad \ldots; \quad \mathbf{e}_V = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}$$

$\mathbf{e}_i$ is the "one hot" vector for the ith word in $\mathcal{V}$.

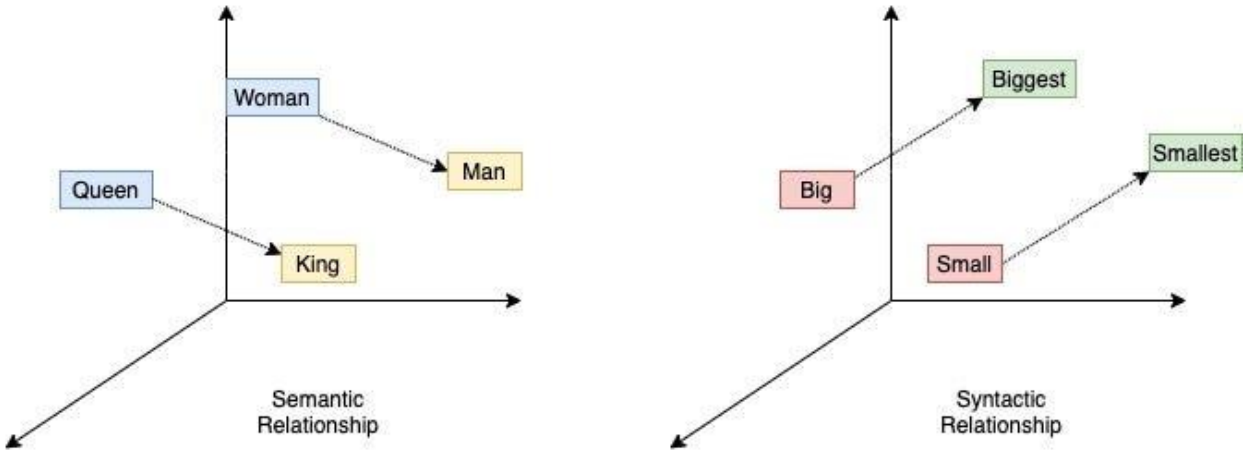
A neural language model starts by "looking up" each word by multiplying its one hot vector by a matrix $\underset{V \times d}{\mathbf{M}}$; $\mathbf{e}_v^\top \mathbf{M} = \mathbf{m}_v$, the "embedding" of v.

$\mathbf{M}$ becomes part of the parameters ($\boldsymbol{\theta}$).

Sequences of Word Vectors

Given a word sequence $\langle v_1, v_2, \ldots, v_k \rangle$, we transform it into a sequence of word vectors,

$$\mathbf{m}_{v_1}, \mathbf{m}_{v_2}, \ldots, \mathbf{m}_{v_k}$$



Adding Layers

- Neural networks are built by composing functions, a mix of
  - Affine, $\boldsymbol{v}' = \boldsymbol{W}\boldsymbol{v} + \boldsymbol{b}$ (note that the dimensionality of $\boldsymbol{v}$ and $\boldsymbol{v}'$ might be different)
  - Nonlinearity, e.g.,
    - rectified linear ("relu") units $v_i' = \max(0, v_i)$
    - elementwise hyperbolic tangent $v_i' = \tanh(v_i) = \dfrac{e^{v_i} - e^{-v_i}}{e^{v_i} + e^{-v_i}}$
    - softmax $v_i' = \exp\{ v_i \} / \sum_j \exp\{ v_j \}$
  - More complex components (composed of the above operations):
    - Convolutional layers
    - Recurrent NNs
    - Attention

Summary so far

- language models utilities
  - Generation, evaluation of fluency, few-shot prediction (GPT3), …
- N-gram language models
  - Unigram LM
  - N-gram LM
- Neural language models:
  - Embedding: one-hot vectors -> embedding vectors
  - Neural networks

Outline

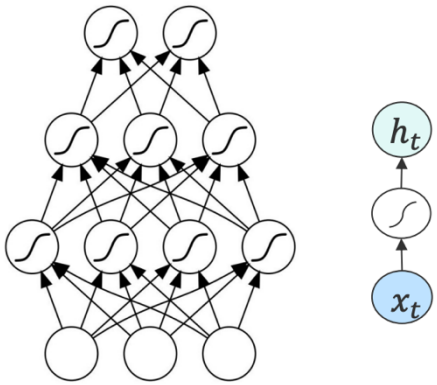
- Recurrent Networks (RNNs)
  - Long-range dependency, vanishing gradients
  - LSTM
  - RNNs in different forms
- Attention Mechanisms
  - (Query, Key, Value)
  - Attention on Text and Images
- Transformers: Multi-head Attention
  - Transformer
  - BERT

Outline

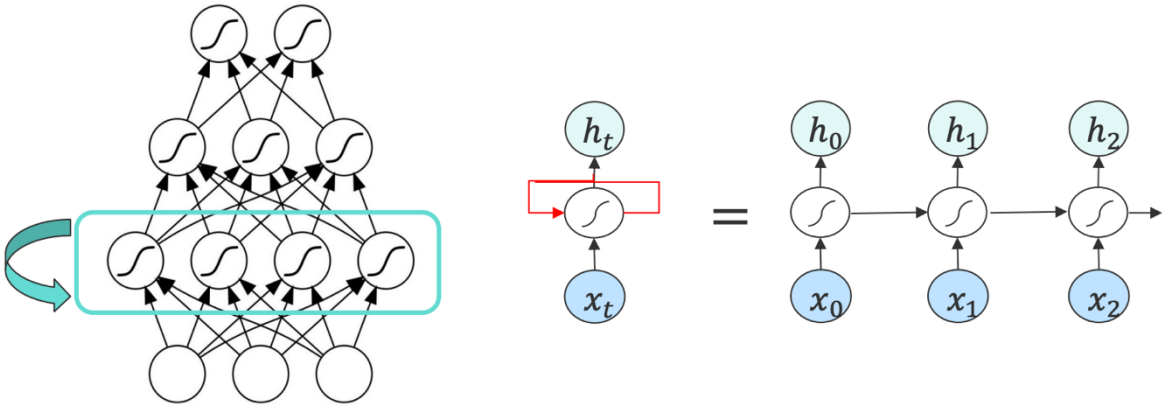
- Recurrent Networks (RNNs)
  - Long-range dependency, vanishing gradients
  - LSTM
  - RNNs in different forms
- Attention Mechanisms
  - (Query, Key, Value)
  - Attention on Text and Images
- Transformers: Multi-head Attention
  - Transformer
  - BERT

ConvNets v.s. Recurrent Networks (RNNs)

- Spatial Modeling *vs.* Sequential Modeling
- Fixed *vs.* variable number of computation steps.



The output depends ONLY on the current input



The hidden layers and the output additionally depend on previous states of the hidden layers

RNNs in Various Forms

One to One

Image classification

One to Many

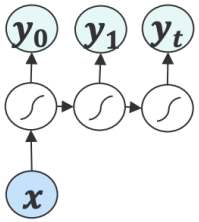
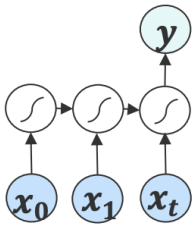


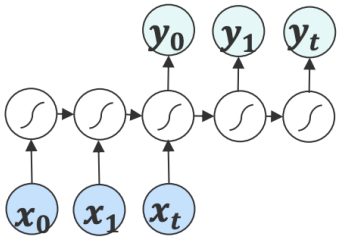
Image captioning

Many to One



Sentence sentiment analysis / Video recognition

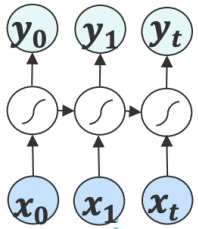
Many to Many



Machine Translation

(Sequence-to-sequence)

Many to Many

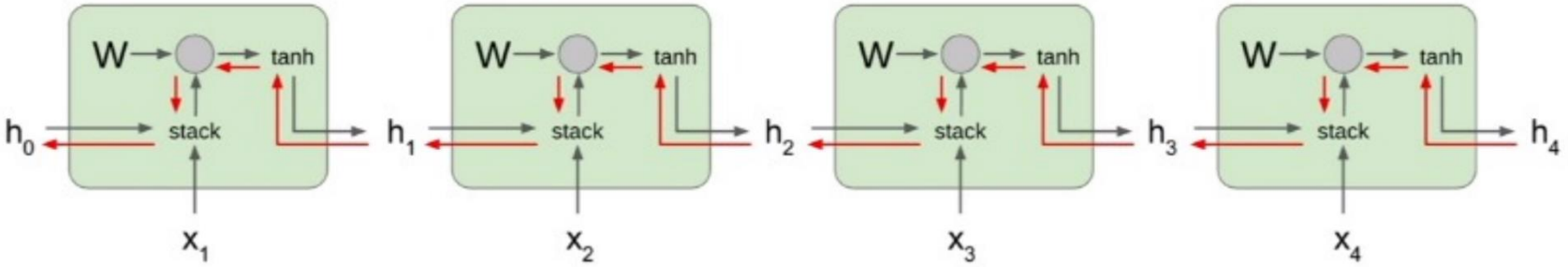


Named Entity Recognition

(Sequence tagging)

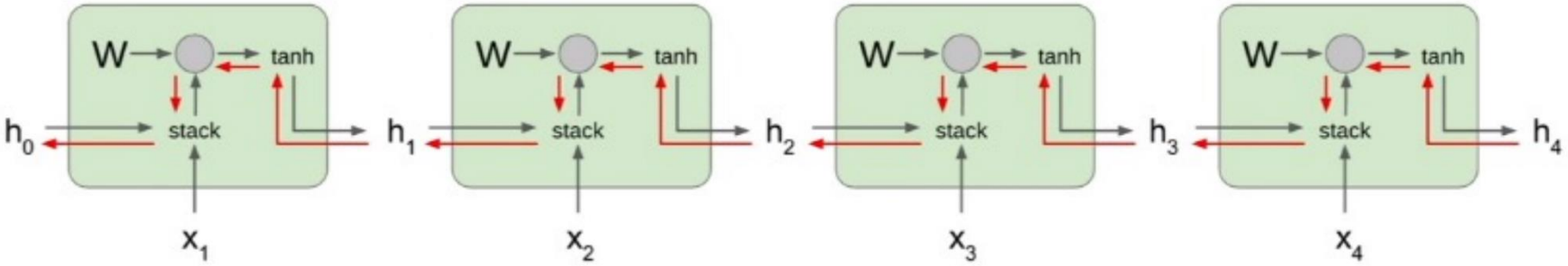
Vanishing / Exploding Gradients in RNNs

$$\boldsymbol{h}_t = tanh(W^{hh}\boldsymbol{h}_{t-1} + W^{hx}\boldsymbol{x}_t)$$



Vanishing / Exploding Gradients in RNNs

$$\boldsymbol{h}_t = tanh(W^{hh}\boldsymbol{h}_{t-1} + W^{hx}\boldsymbol{x}_t)$$



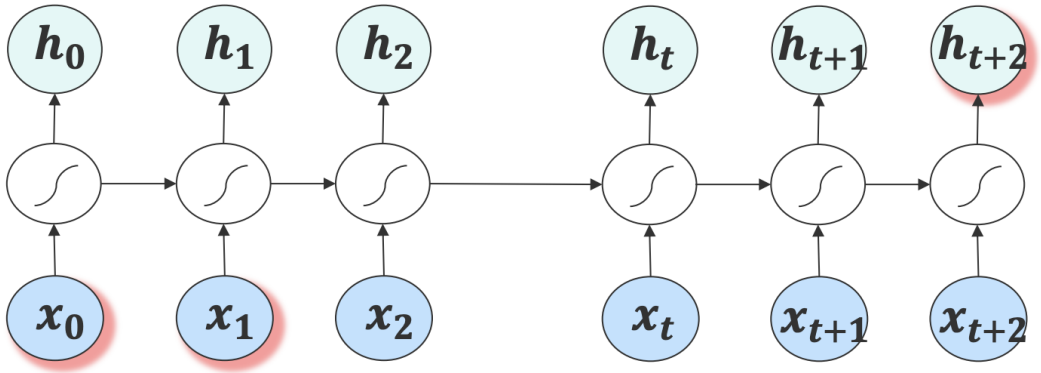
Computing gradient of h_0 involves many factors of W (and repeated tanh)

Largest singular value > 1: **Exploding gradients** → **Gradient clipping**: Scale gradient if its norm is too big

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
  grad *= (threshold / grad_norm)
```

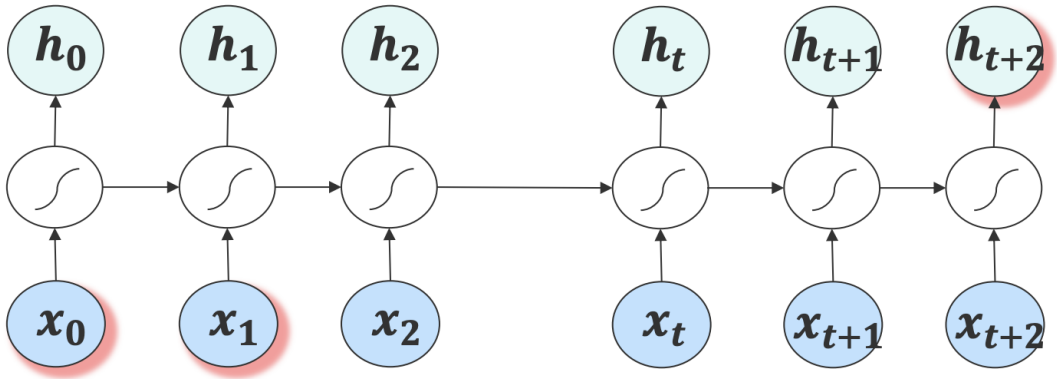
Largest singular value < 1: **Vanishing gradients**

Long-term Dependency Problem



I live in France and I know _____

Long-term Dependency Problem

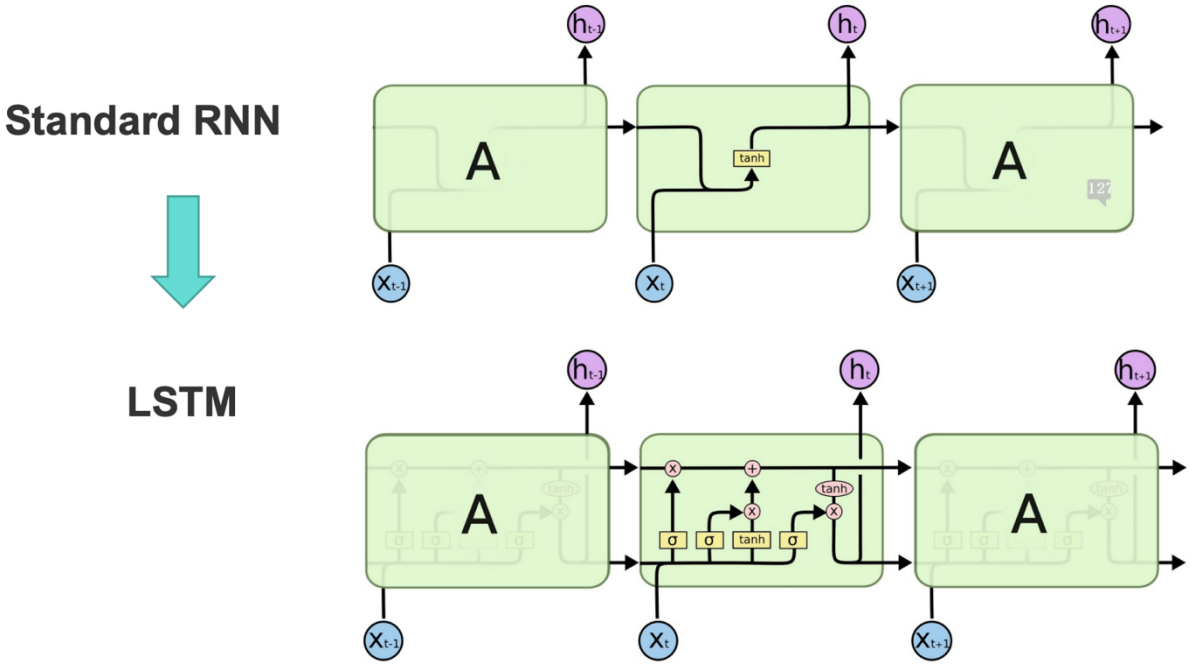


I live in France and I know ___French___

I live in France, a beautiful country, and I know ___French___

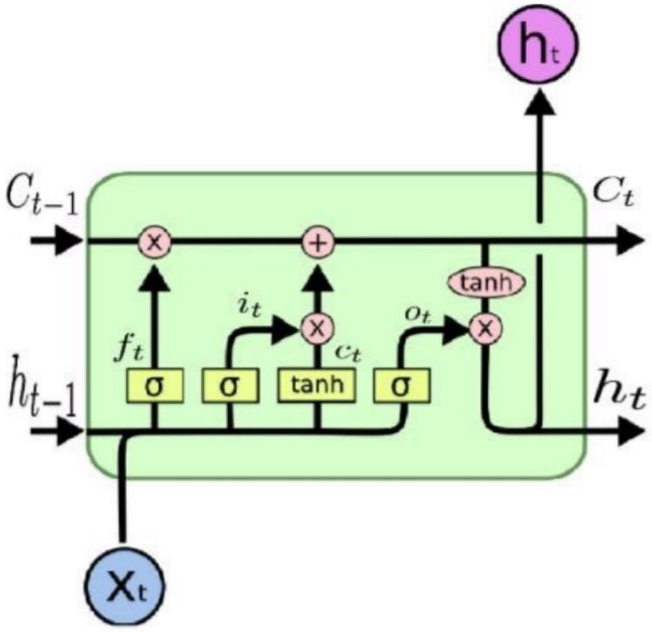
Long Short Term Memory (LSTM)

- LSTMs are designed to explicitly alleviate the long-term dependency problem [Horchreiter & Schmidhuber (1997)]



Long Short Term Memory (LSTM)

- Gate functions make decisions of reading, writing, and resetting information

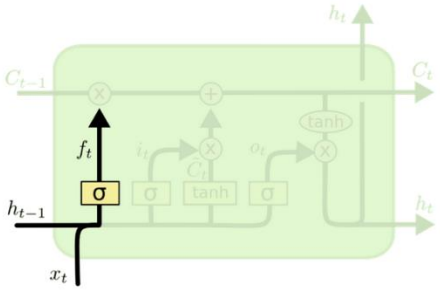


- Forget gate: whether to erase cell (reset)
- Input gate: whether to write to cell (write)
- Output gate: how much to reveal cell (read)

15

Long Short Term Memory (LSTM)

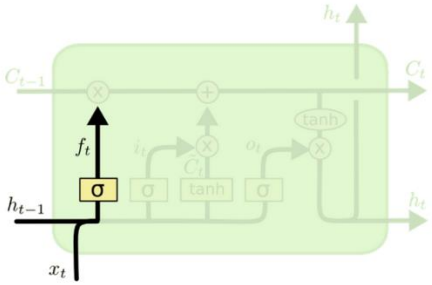
- Forget gate: decides what must be removed from $\boldsymbol{h_{t-1}}$



$$\boldsymbol{f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)}$$

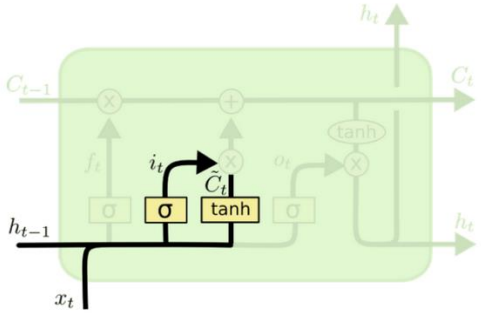
Long Short Term Memory (LSTM)

- Forget gate: decides what must be removed from $\boldsymbol{h_{t-1}}$



$$\boldsymbol{f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)}$$

- Input gate: decides what new information to store in the cell

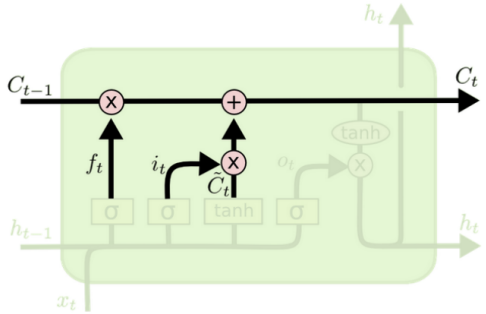


$$\boldsymbol{i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)}$$

$$\boldsymbol{\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)}$$

Long Short Term Memory (LSTM)

- Update cell state:

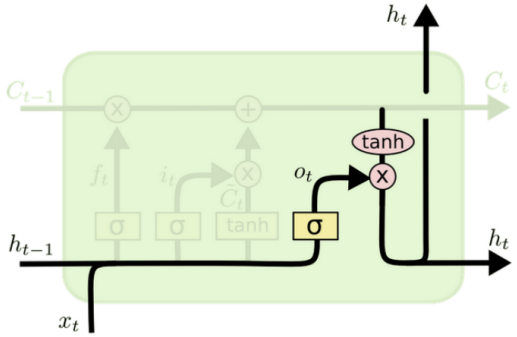


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

forgetting unneeded things

scaling the new candidate values by how much we decided to update each state value.

- Output gate: decides what to output from our cell state



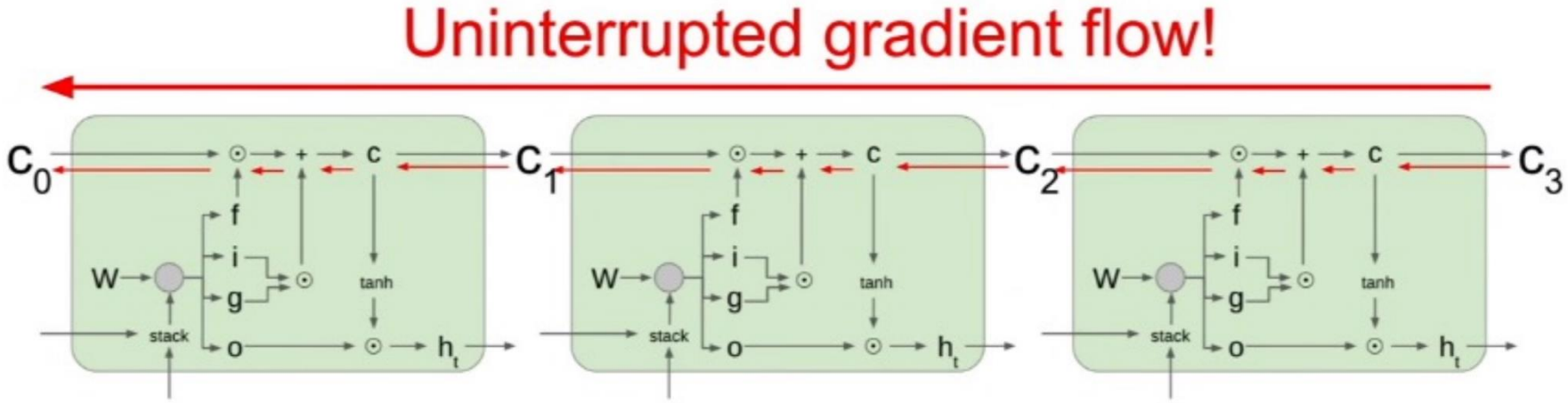
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

sigmoid decides what parts of the cell state we're going to output

19

Backpropagation in LSTM



- No multiplication with matrix W during backprop
- Multiplied by different values of forget gate -> less prone to vanishing/exploding gradient

Outline

- Recurrent Networks (RNNs)
  - Long-range dependency, vanishing gradients
  - LSTM
  - RNNs in different forms
- Attention Mechanisms
  - (Query, Key, Value)
  - Attention on Text and Images
- Transformers: Multi-head Attention
  - Transformer
  - BERT

Attention: Examples

- Chooses which features to pay attention to

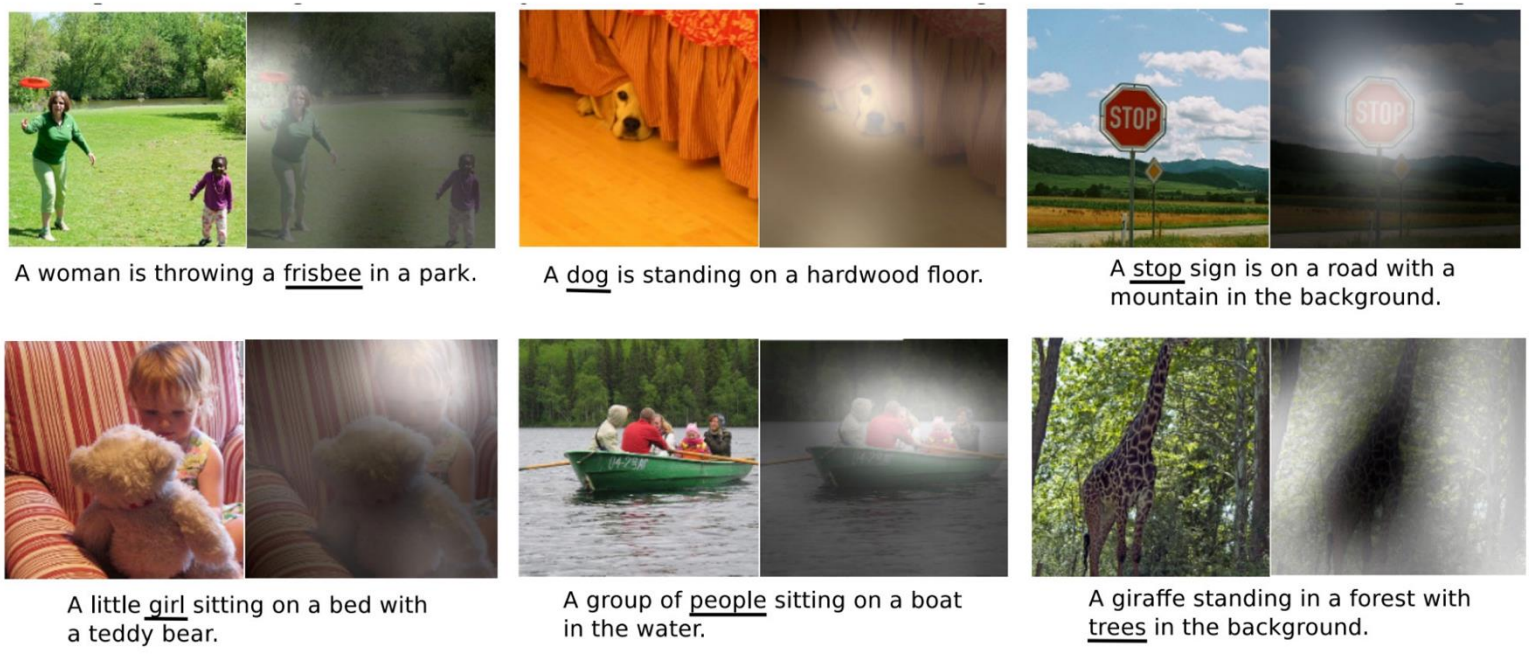
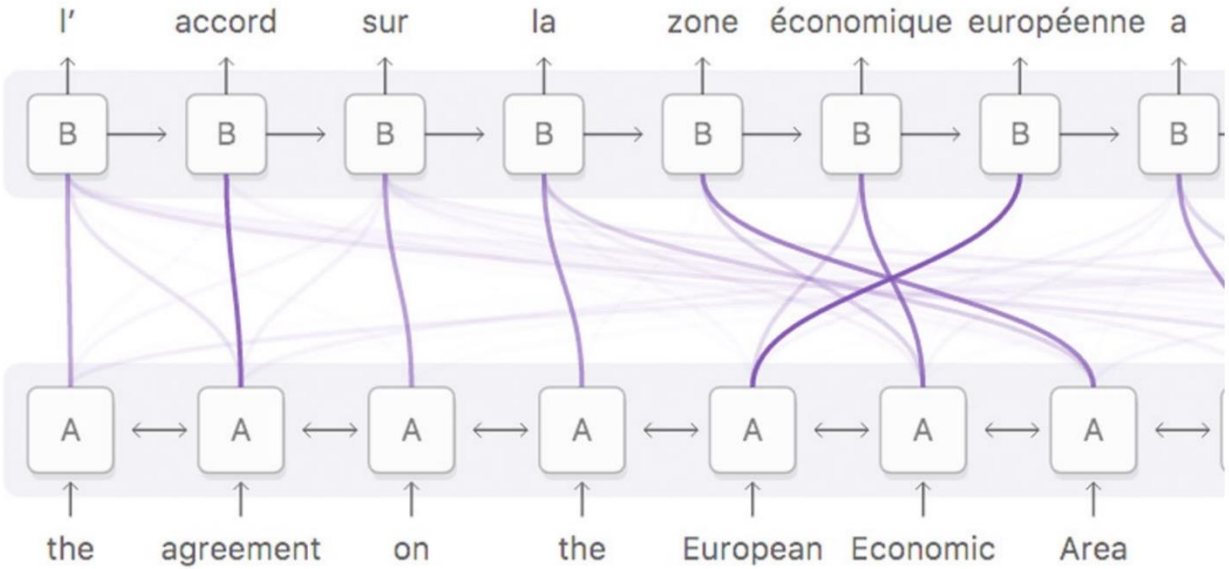


Image captioning [Show, attend and tell. Xu et al. 15]

27

Attention: Examples

- Chooses which features to pay attention to



Machine Translation

Figure courtesy: Olah & Carter, 2016

Why Attention?

- Long-range dependencies
  - Dealing with gradient vanishing problem

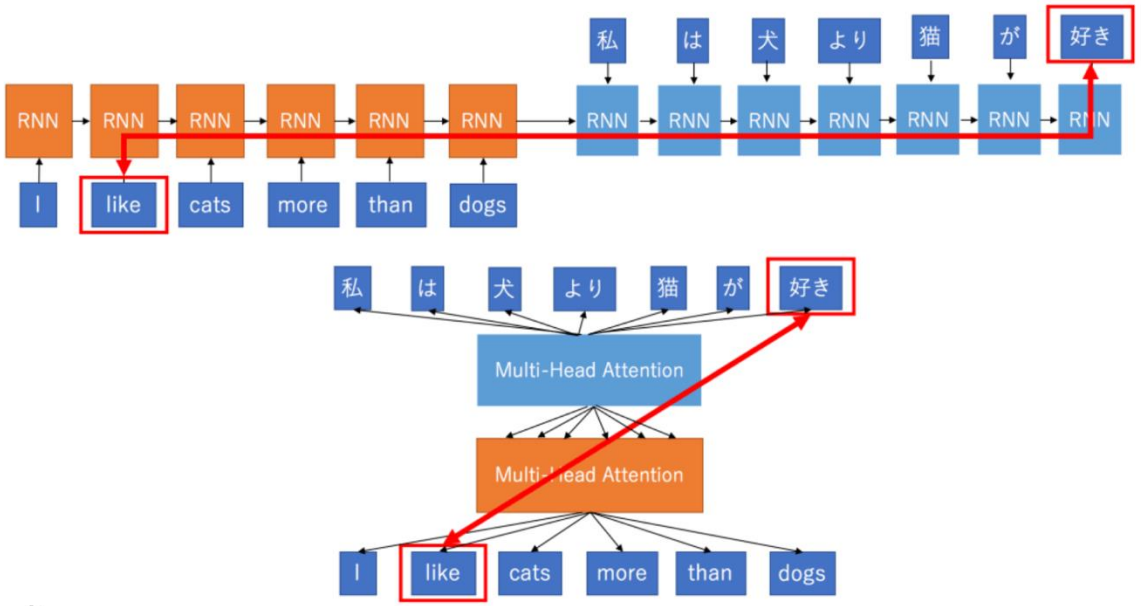


Figure courtesy: keitakurita

30

Why Attention?

- Long-range dependencies
  - Dealing with gradient vanishing problem
- Fine-grained representation instead of a single global representation
  - Attending to smaller parts of data: patches in images, words in sentences

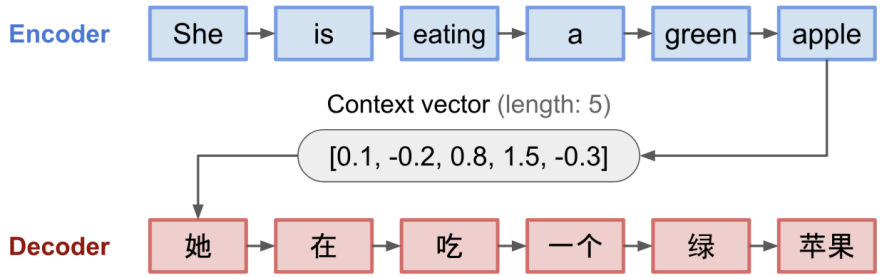


Figure courtesy: Lilian Weng

31

Why Attention?

- Long-range dependencies
  - Dealing with gradient vanishing problem
- Fine-grained representation instead of a single global representation
  - Attending to smaller parts of data: patches in images, words in sentences
- Improved Interpretability

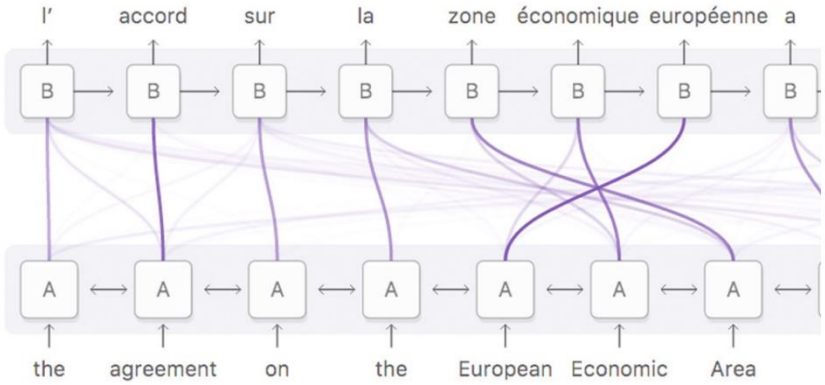


Figure courtesy: Olah & Carter, 2016

32

Attention Computation

- Encode each token in the input sentence into vectors
- When decoding, perform a linear combination of these vectors, weighted by "attention weights"
  - $\boldsymbol{a} = \text{softmax}(\boldsymbol{alignment\_scores})$

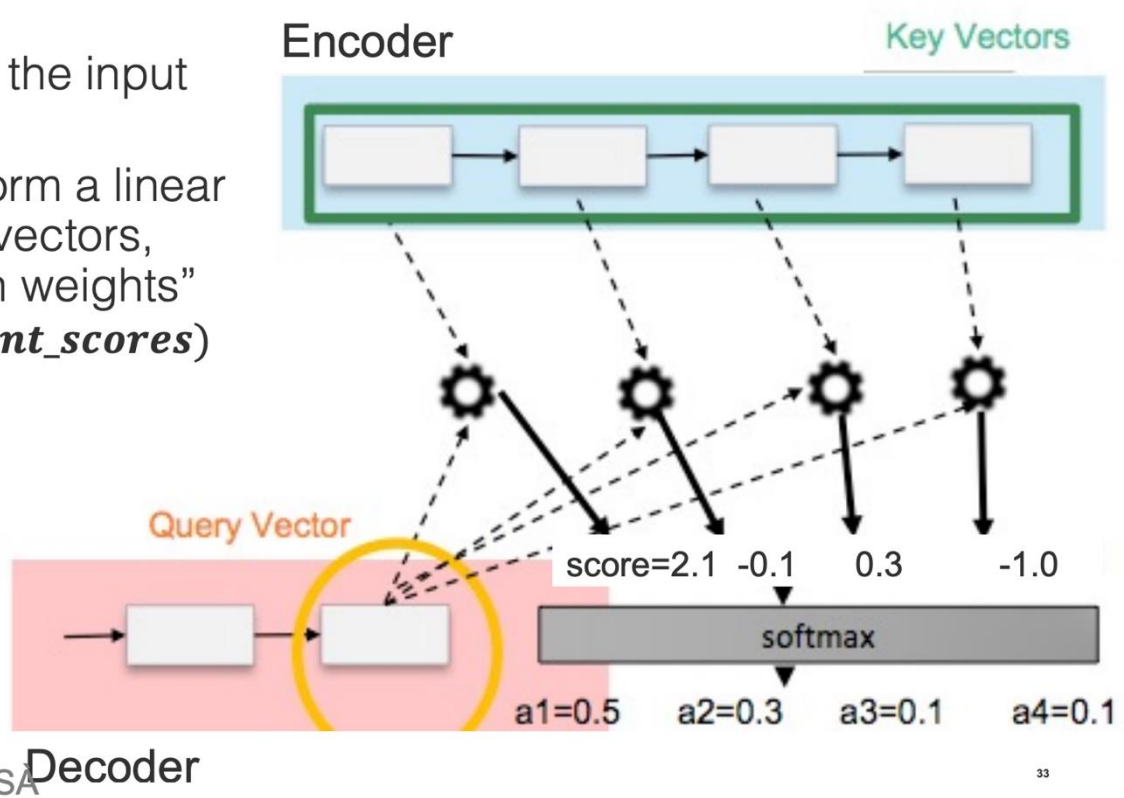
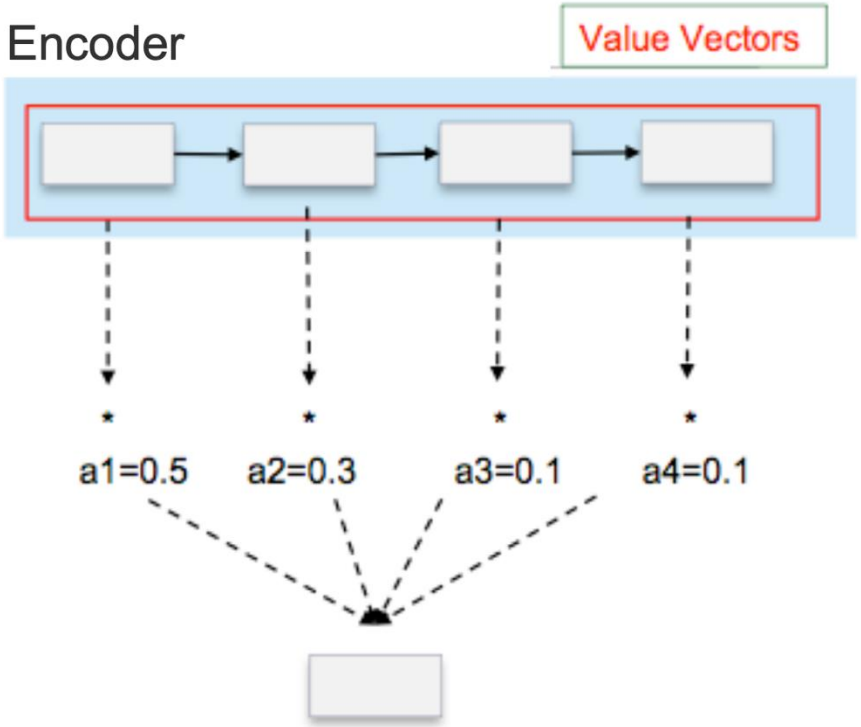


Figure courtesy: MARTA R. COSTA-JUSSÀ

Attention Computation (cont'd)

- Combine together value by taking the weighted sum



- Query: decoder state
- Key: all encoder states
- Value: all encoder states

35

Attention Variants

- Popular attention mechanisms with different alignment score functions

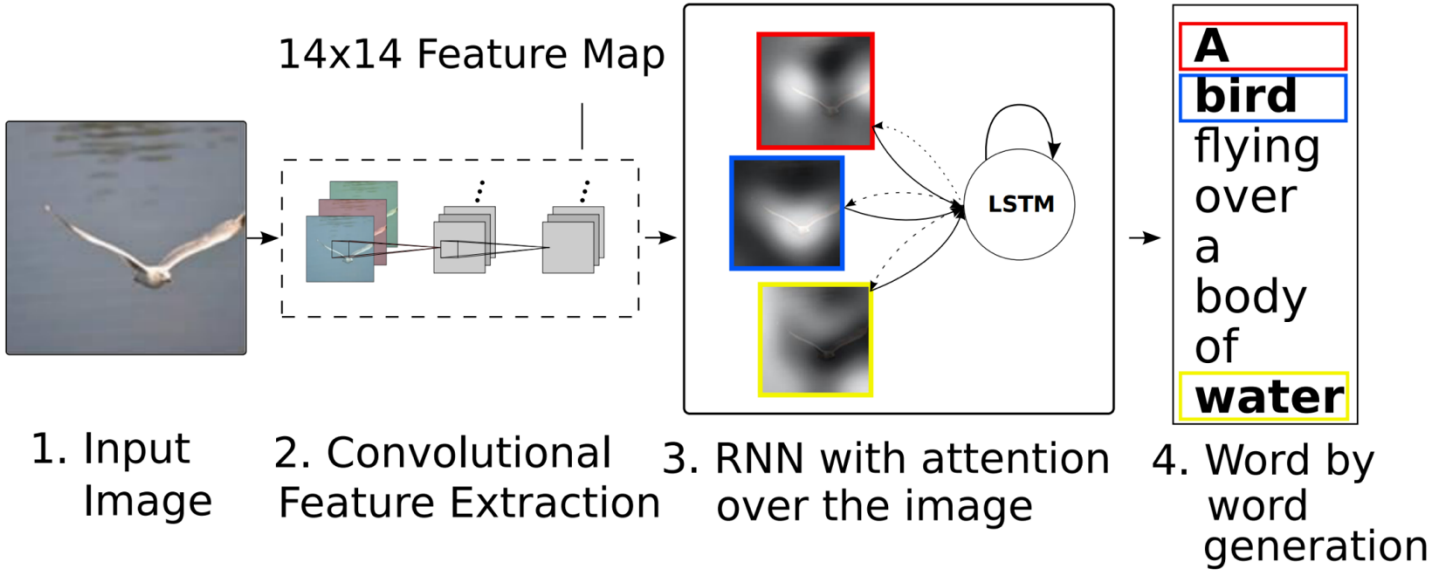
Alignment score = f(Query, Keys)

- Query: decoder state s_t
- Key: all encoder states h_i
- Value: all encoder states h_i

Name	Alignment score function	Citation
Content-base attention	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \text{cosine}[\boldsymbol{s}_t, \boldsymbol{h}_i]$	Graves2014
Additive(*)	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\boldsymbol{s}_t; \boldsymbol{h}_i])$	Bahdanau2015
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a \boldsymbol{s}_t)$ Note: This simplifies the softmax alignment to only depend on the target position.	Luong2015
General	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^\top \mathbf{W}_a \boldsymbol{h}_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer.	Luong2015
Dot-Product	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \boldsymbol{s}_t^\top \boldsymbol{h}_i$	Luong2015
Scaled Dot-Product(^)	$\text{score}(\boldsymbol{s}_t, \boldsymbol{h}_i) = \frac{\boldsymbol{s}_t^\top \boldsymbol{h}_i}{\sqrt{n}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.	Vaswani2017

Courtesy: Lilian Weng

Attention on Images – Image Captioning



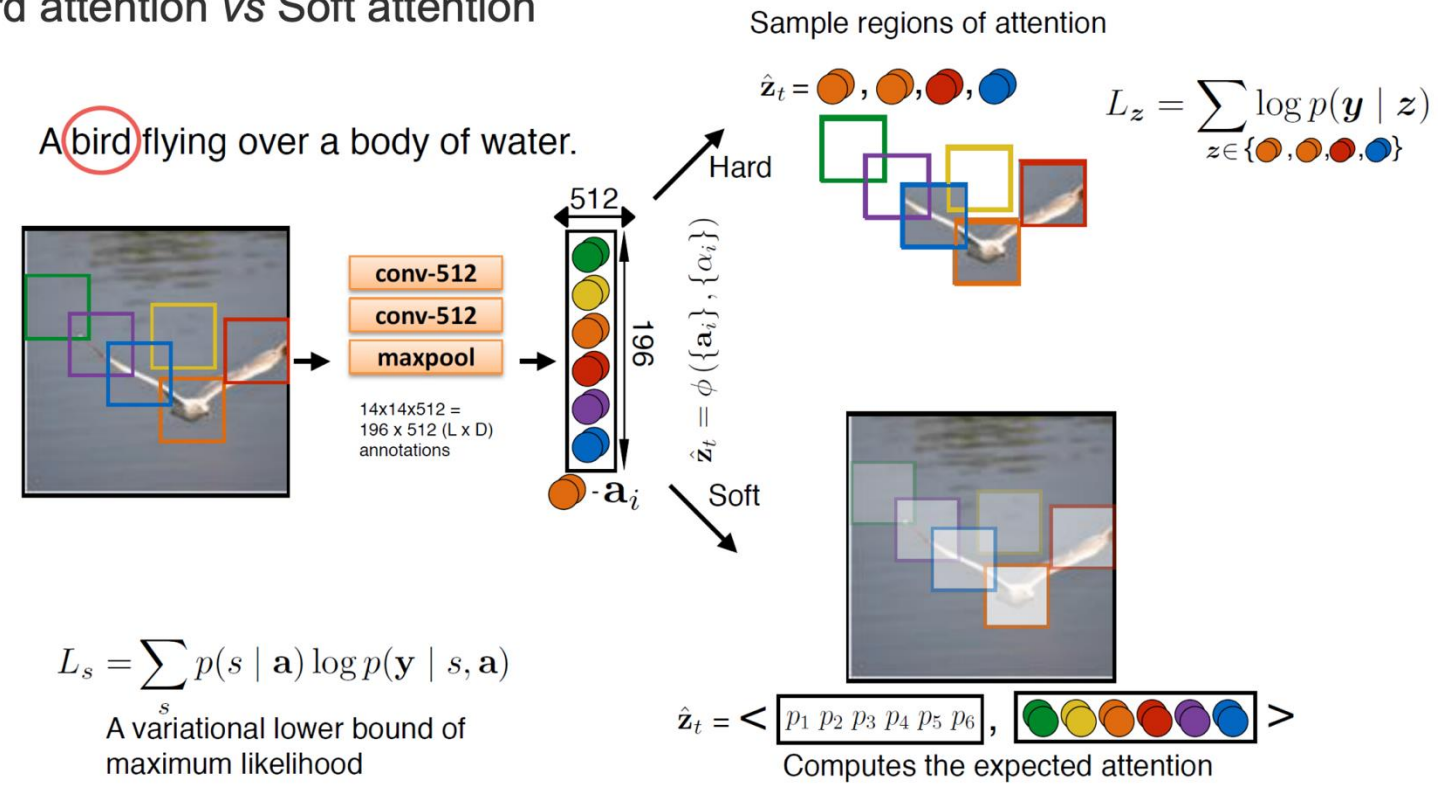
- Query: decoder state
- Key: visual feature maps
- Value: visual feature maps

[Show, attend and tell. Xu et al. 15]

37

Attention on Images – Image Captioning

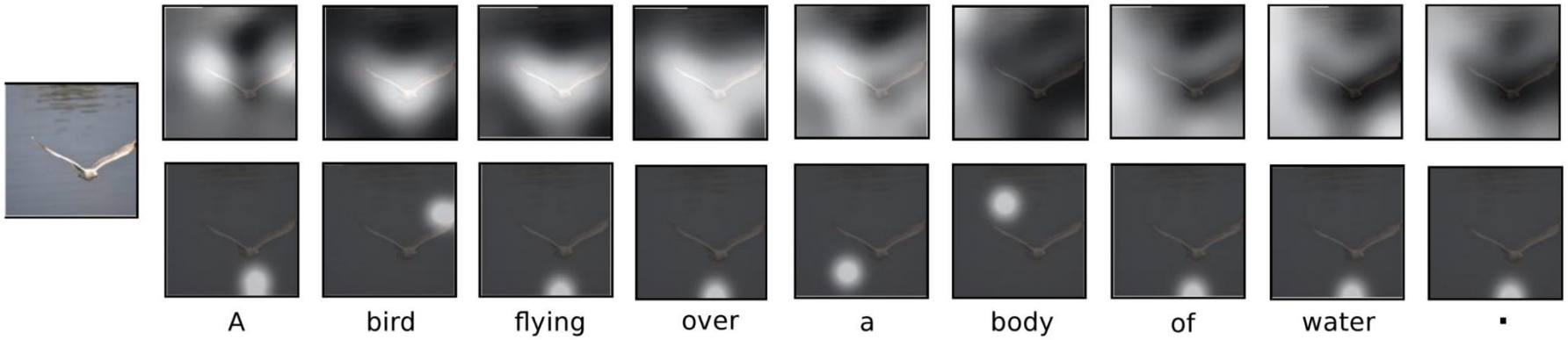
Hard attention *vs* Soft attention



38

Attention on Images – Image Captioning

Hard attention *vs* Soft attention



39

Attention on Images – Image Paragraph Generation

- Generate a long paragraph to describe an image
  - Long-term visual and language reasoning
  - Contentful descriptions -- ground sentences on visual features

This picture is taken for three baseball players on a field. The man on the left is wearing a blue baseball cap. The man has a red shirt and white pants. The man in the middle is in a wheelchair and holding a baseball bat. Two men are bending down behind a fence. There are words band on the fence.

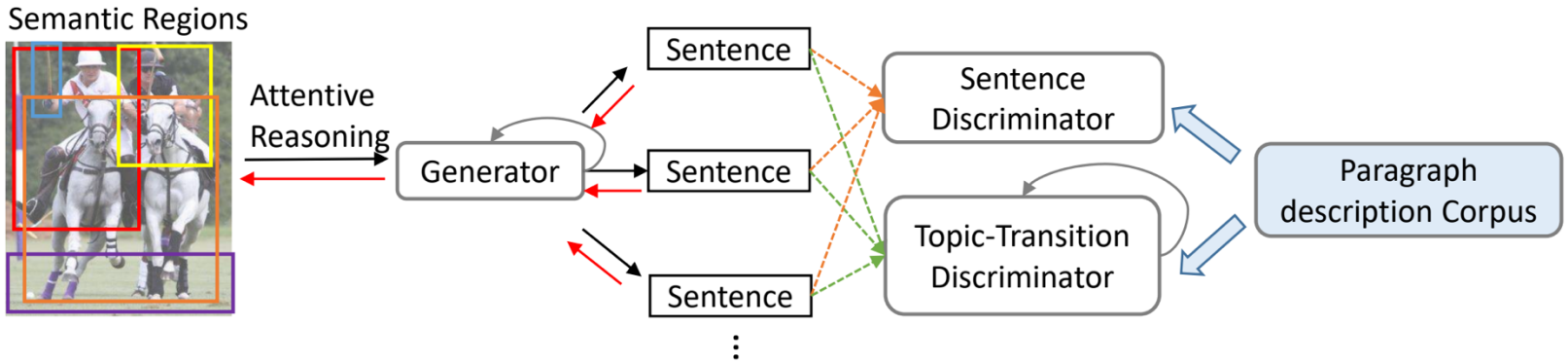
A tennis player is attempting to hit the tennis ball with his left foot hand. He is holding a tennis racket. He is wearing a white shirt and white shorts. He has his right arm extended up. There is a crowd of people watching the game. A man is sitting on the chair.

A couple of zebra are standing next to each other on dirt ground near rocks. There are trees behind the zebras. There is a large log on the ground in front of the zebra. There is a large rock formation to the left of the zebra. There is a small hill near a small pond and a wooden log. There are green leaves on the tree.

40

Image Mining - Application

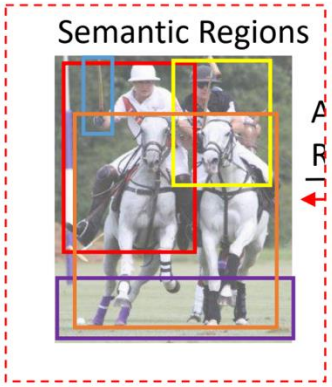
Attention on Images – Image Paragraph Generation



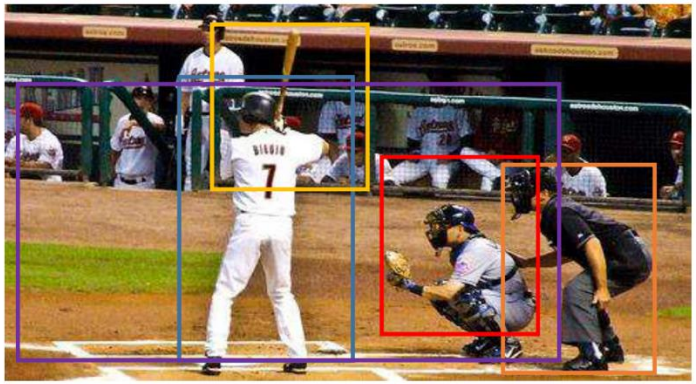
[Recurrent Topic-Transition GAN for Visual Paragraph Generation. Liang et al. 2017]

41

Attention on Images – Image Paragraph Generation



Semantic region detection & captioning

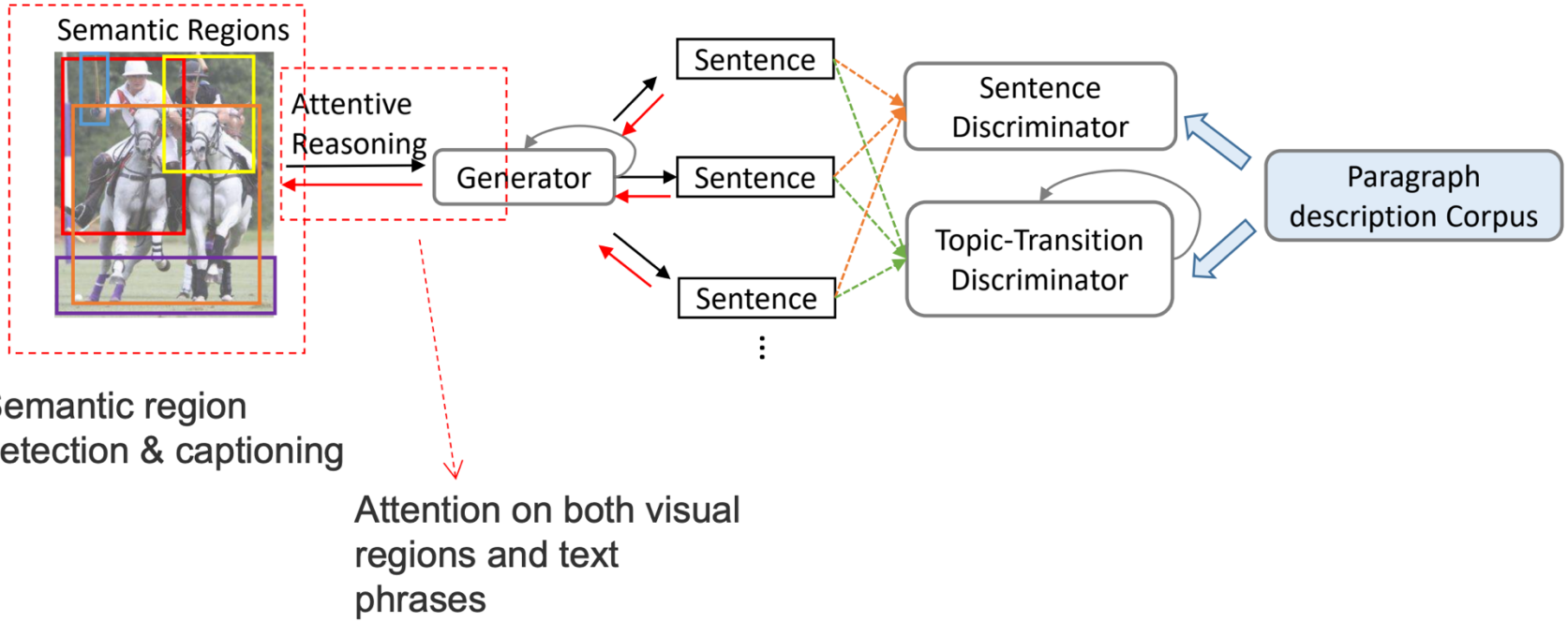


Local Phrases

- people playing baseball
- a man wearing white shirt and pants
- man holding a baseball bat
- person wearing a helmet in the field
- a man bending over

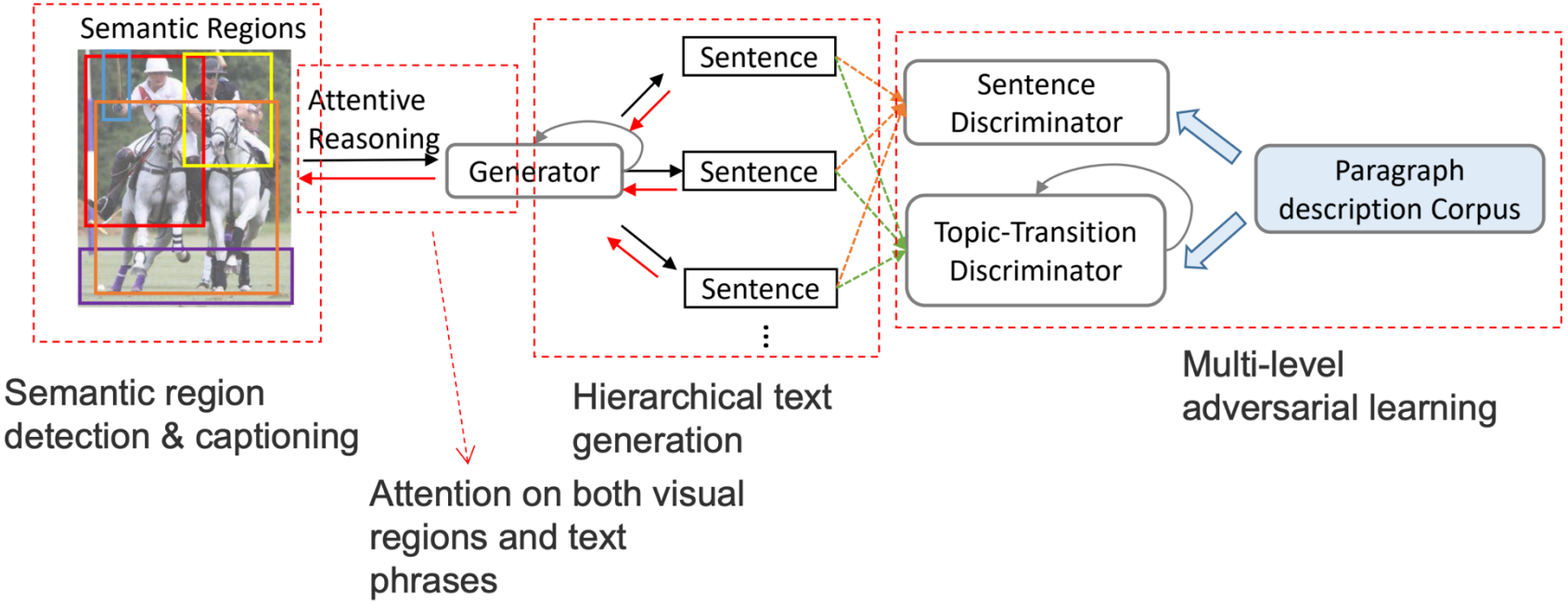
42

Attention on Images – Image Paragraph Generation



43

Attention on Images – Image Paragraph Generation



45

Transformers – Multi-head (Self-)Attention

- State-of-the-art Results by Transformers
  - [Vaswani et al., 2017] Attention Is All You Need
    - Machine Translation
  - [Devlin et al., 2018] BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
    - Pre-trained Text Representation
  - [Radford et al., 2019] Language Models are Unsupervised Multitask Learners
    - Language Models

48

Image Mining - Application

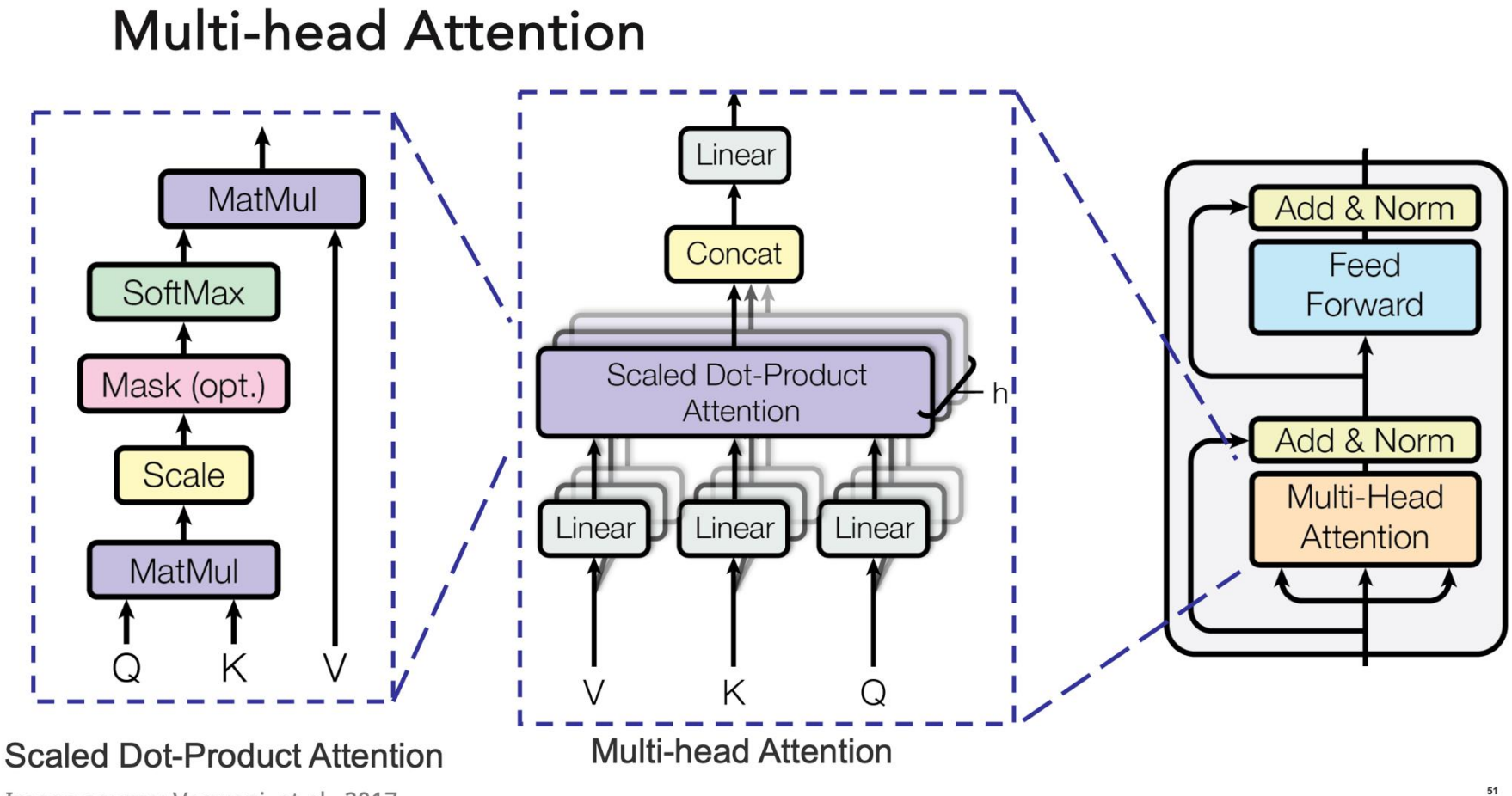


Image source: Vaswani, et al., 2017

Multi-head Attention in Encoders and Decoders

Transformer

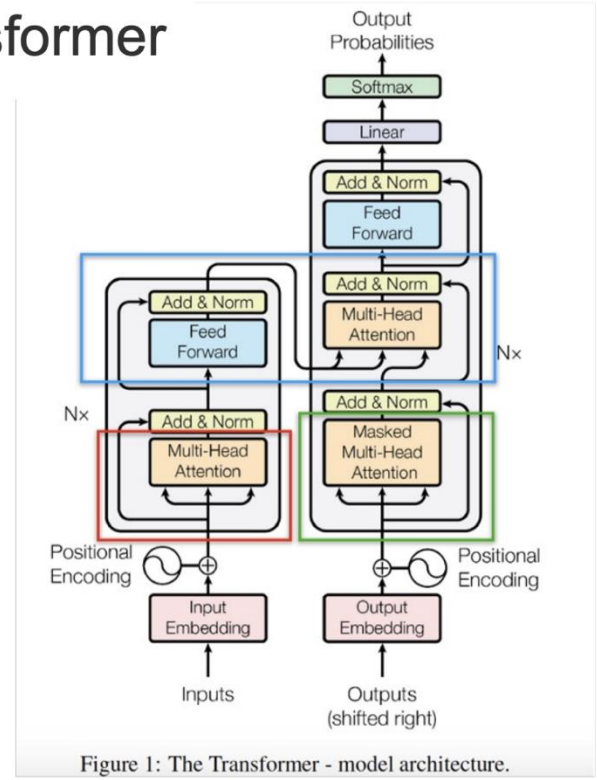


Figure 1: The Transformer - model architecture.

Image source: Bgg

encoder self attention

1. Multi-head Attention
2. Query=Key=Value

decoder self attention

1. Masked Multi-head Attention
2. Query=Key=Value

encoder-decoder attention

1. Multi-head Attention
2. Encoder Self attention=Key=Value
3. Decoder Self attention=Query

53

Question Time!